BACKEND · REAL-TIME COMMUNICATION

# Real-time with Socket.IO

Why HTTP alone cannot build a chat app, what Socket.IO adds on top of WebSocket, how events carry data between client and server, and what breaks when you run more than one server.

Node.js | Express | Socket.IO v4 | Redis adapter

---

## 01 The Problem

In HTTP the server can never speak first

Everything you have built so far with Express follows one rule: the client sends a request, the server sends a response, and the exchange is over. The server only talks when it is asked something.

That works for loading a page or saving a form. It fails the moment new data appears on the server and the client has no reason to ask for it.

> KEY CONCEPT
>
> **Request–response** is a pull model: data moves only when the client pulls it. **Real-time** needs a push model: the server sends data the moment it exists, without waiting to be asked.

### The chat example

Riya sends a message to Aman. The server receives it and saves it. Now what?

```
Riya → POST /messages → Server
    ↓
Server saves the message ✓
    ↓
Server wants to tell Aman
    ↓
✗ No open request from Aman, so there is nothing to respond to
    ↓
Aman sees the message only after he refreshes
```

## Workaround 1: Polling

The client asks `GET /messages` every few seconds. Most of those requests return nothing new, each one carries full HTTP headers, and a message still arrives late by up to one polling interval.

## Workaround 2: Long polling

The client sends a request and the server holds it open until there is something to send. When the response arrives, the client immediately sends the next request. Messages arrive quickly, but every message still costs a full HTTP request, and the server is holding thousands of hanging requests.

| What | Polling | Long polling | What we actually need |
|---|---|---|---|
| **Who starts** | Client, on a timer | Client, again after every response | Either side, any time |
| **Delay** | Up to one interval | Low | Close to zero |
| **Wasted work** | Many empty responses | Headers on every message | A few bytes per message |
| **Connection** | New one each time | Reopened after each message | One, kept open |

💡 Both workarounds are the client **pretending** the server can push. Gmail chat and early Facebook chat ran on long polling, and the pain of running it at scale is why WebSocket was standardised in 2011.

Why can a plain Express server not deliver a new chat message to Aman on its own?

A — Express is too slow to send messages instantly

B — In HTTP the server can only respond to a request, and Aman has not sent one

What is the main cost of long polling compared with a persistent connection?

A — Every message still needs a full HTTP request with headers

B — Messages are delayed by the polling interval

## 02 The Solution

WebSocket gives the pipe, Socket.IO makes it usable

### WebSocket

WebSocket is a protocol that keeps **one connection open** between client and server. Once it is open, either side can send a message at any time, with only a few bytes of overhead per message.

It starts as a normal HTTP request carrying an `Upgrade: websocket` header. The server replies with status `101 Switching Protocols`, and from then on the same TCP connection speaks WebSocket instead of HTTP.

### Why not use raw WebSocket directly?

Raw WebSocket only gives you `send(string)` and an `onmessage` handler. Everything a real app needs is left to you:

- The connection drops on a network change, and nothing reconnects it.
- There is no way to say "send this to everyone in room 42".
- Every message is one unnamed string, so you must invent your own message format.

- You cannot tell whether the other side received what you sent.
- Some corporate proxies block WebSocket, and those users simply cannot connect.

> KEY CONCEPT
>
> **Socket.IO** is a library that sits on top of WebSocket and fills those gaps: named **events**, automatic **reconnection**, **rooms** and broadcasting, **acknowledgements**, and a **fallback** to HTTP long polling when WebSocket is blocked.

| What | Raw WebSocket | Socket.IO |
|---|---|---|
| **Message shape** | One string or binary blob | Named event + any JSON data ✓ |
| **Reconnection** | Write it yourself | Built in, with backoff ✓ |
| **Groups of clients** | Keep your own arrays | Rooms and namespaces ✓ |
| **Delivery confirmation** | None | Acknowledgement callbacks ✓ |
| **WebSocket blocked** | Connection fails | Falls back to long polling ✓ |
| **Bundle size and overhead** | Nothing extra ✓ | Client library + small per-packet metadata |

⚠️ **Socket.IO is not WebSocket.** It adds its own packet format on top. A plain WebSocket client cannot connect to a Socket.IO server, and a Socket.IO client cannot connect to a plain WebSocket server. Both sides must use Socket.IO.

## How a Socket.IO connection is established

**Handshake over HTTP**

The client sends a normal HTTP request. The server replies with a session ID ( `sid` ) and its ping settings.

**Start on long polling**

The connection works immediately over HTTP long polling, which gets through almost every network.

**Upgrade to WebSocket**

In the background the client tries to open a WebSocket. If it succeeds, the connection switches to it.

**Connected**

Both sides exchange events. Ping and pong packets detect a dead connection, and the client reconnects on its own.

---

## The one-way alternative: Server-Sent Events (SSE)

Not every real-time feature needs two directions. A notification bell, a live cricket score or an AI answer appearing word by word only needs the **server to push to the client**. For that there is a simpler tool than WebSocket.

> KEY CONCEPT
>
> **Server-Sent Events** is a normal HTTP response that never ends. The client makes one `GET` request, the server replies with `Content-Type: text/event-stream`, keeps the response open, and writes a new chunk of text every time it has something to send. Data flows **only from server to client**.

SERVER (EXPRESS) COPY

```
app.get("/events", (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.flushHeaders();

  // each message is "data: ..." followed by a blank line
  const timer = setInterval(() => {
    res.write(`data: ${JSON.stringify({ time: Date.now() })}\n\n`);
  }, 1000);

  // the client closed the tab: stop writing
  req.on("close", () => clearInterval(timer));
});
```

CLIENT (BUILT INTO THE BROWSER) COPY

```
const source = new EventSource("http://localhost:3000/events");

source.onmessage = (event) => {
  const data = JSON.parse(event.data);
```

```
  console.log(data.time);
};
```

No library is needed on either side. `EventSource` is a browser API, and it reconnects by itself when the connection drops. When the client needs to send something, it uses a normal `fetch` request.

| What | SSE | WebSocket / Socket.IO |
|---|---|---|
| **Direction** | Server → client only | Both ways |
| **Protocol** | Plain HTTP, works with existing routes, auth cookies and proxies | Separate protocol after an upgrade |
| **Data** | Text only | Text and binary |
| **Reconnection** | Built into the browser | Built into Socket.IO, manual in raw WebSocket |
| **Rooms, acknowledgements** | None | Built into Socket.IO |
| **Use it for** | Notifications, live scores, dashboards, AI response streaming | Chat, multiplayer games, collaborative editing |

💡 **How to choose:** if the client only listens, use SSE. If the client also sends frequently and you need rooms or acknowledgements, use Socket.IO. The word-by-word answers in ChatGPT-style apps are SSE, because the user sends one prompt and then only receives.

⚠️ **Two SSE limits to know.** `EventSource` cannot send custom headers, so a token in an `Authorization` header will not work; use a cookie. And over HTTP/1.1 a browser allows only 6 open connections per domain, so several tabs each holding an SSE stream can block your other requests. HTTP/2 removes that limit.

Can a browser's built-in `new WebSocket(url)` connect to a Socket.IO server?

A — Yes, Socket.IO is just a WebSocket server with a nicer API

B — No, Socket.IO uses its own protocol on top, so the client must be Socket.IO too

By default, which transport does a Socket.IO connection start with?

A — HTTP long polling, then it upgrades to WebSocket

B — WebSocket, then it falls back to polling on failure

## 03 Implementing Socket.IO

One package on the server, one on the client, attached to your existing Express app

**STEP 1 — INSTALL**

Socket.IO ships as two packages. The server one goes in your backend, the client one in your frontend.

TERMINAL COPY

```
# backend
npm install express socket.io

# frontend
npm install socket.io-client
```

**STEP 2 — CREATE THE SERVER**

Socket.IO needs the raw Node HTTP server, because it has to handle the upgrade request itself. So you wrap the Express app in `createServer` and give that server to Socket.IO.

SERVER.JS COPY

```
import express from "express";
import { createServer } from "node:http";
import { Server } from "socket.io";
```

```
const app = express();
const httpServer = createServer(app);

const io = new Server(httpServer, {
  cors: { origin: "http://localhost:5173" },
});

io.on("connection", (socket) => {
  console.log("connected:", socket.id);

  socket.on("disconnect", (reason) => {
    console.log("disconnected:", socket.id, reason);
  });
});

httpServer.listen(3000, () => {
  console.log("listening on :3000");
});
```

| PIECE | VALUE USED | WHAT IT DOES |
|---|---|---|
| **createServer(app)** | httpServer | The real Node HTTP server. Express handles normal routes on it, Socket.IO handles its own path ( `/socket.io/` ). |
| **new Server()** | io | The Socket.IO server. It represents **all** connected clients. |
| **cors.origin** | localhost:5173 | Which frontend origin may connect. Needed whenever frontend and backend run on different ports or domains. |
| **"connection"** | (socket) => {} | Runs once for every client that connects. All listeners for that client are registered inside it. |
| **socket.id** | random string | Unique ID for this connection. It changes on every reconnect, so never use it as a user ID. |

⚠️ **Call `httpServer.listen()`, not `app.listen()`.** `app.listen()` creates a second, separate HTTP server that Socket.IO is not attached to. Your REST routes will work and every socket connection will fail.

KEY CONCEPT

> `io` is the whole server: use it to talk to everyone or to a room. `socket` is one single client's connection: use it to listen to that client and to reply to that client.

## STEP 3 — CONNECT FROM THE CLIENT

SOCKET.JS (FRONTEND) COPY

```
import { io } from "socket.io-client";

export const socket = io("http://localhost:3000");

socket.on("connect", () => {
  console.log("connected with id:", socket.id);
});

socket.on("disconnect", (reason) => {
  console.log("disconnected:", reason);
});

socket.on("connect_error", (err) => {
  console.log("could not connect:", err.message);
});
```

💡 Create the client socket **once** in its own file and import it wherever you need it. Calling `io()` inside a React component creates a new connection on every render.

## VERIFY

Start the backend, open the frontend, and check both consoles. The same ID must appear on both sides. Stop the server and start it again: the client reconnects by itself and gets a new ID.

```
server  connected: Xk3f9aQ2LmB7dE1pAAAB
browser connected with id: Xk3f9aQ2LmB7dE1pAAAB
same ID on both sides → the connection is working ✓
```

Your REST routes work but the socket never connects. Which line is the usual cause?

A — You called app.listen(3000) instead of httpServer.listen(3000)

B — You imported createServer from node:http

Why should `socket.id` not be used to identify a user?

A — It is the same for every client on the server

B — It identifies one connection and changes on every reconnect

## 04 What are Events?

A name plus some data, sent over the open connection

In REST you tell the server what you want with a method and a URL: `POST /messages`. On a socket there are no URLs, because there is only one open connection. So you label each message with a **name** instead. That named message is an event.

> KEY CONCEPT
>
> An **event** is a name (a string you choose) plus data. One side calls `emit(name, data)` to send it. The other side has already called `on(name, handler)` to listen for that exact name. If the names do not match, nothing happens and no error is thrown.

| What | REST | Socket.IO |
|---|---|---|
| **How you address a handler** | Method + URL | Event name |
| **Register a handler** | app.post("/messages", fn) | socket.on("message:send", fn) |
| **Trigger it** | fetch("/messages", …) | socket.emit("message:send", data) |
| **Who can trigger** | Only the client | Client or server |

### Emit and listen

CLIENT COPY

```
// send an event to the server
socket.emit("message:send", { roomId: "chat:42", text: "Hello!" });

// listen for an event from the server
socket.on("message:new", (message) => {
  console.log(message.from, message.text);
});
```

SERVER COPY

```
io.on("connection", (socket) => {
  // listen for an event from this client
  socket.on("message:send", (payload) => {
    console.log(payload.roomId, payload.text);
  });

  // send an event to this client
  socket.emit("welcome", { serverTime: Date.now() });
});
```

### What data can an event carry?

Anything that can be turned into JSON: objects, arrays, strings, numbers, booleans. You can pass several arguments, and binary data such as a `Buffer` or `ArrayBuffer` is supported too. Functions, class instances, `Map` and `Set` do not survive the trip.

### Built-in events

A few event names are fired by Socket.IO itself. You listen to them, you never emit them.

| EVENT | FIRED ON | WHEN |
|---|---|---|
| **connection** | server · io | A new client has connected. The handler receives that client's `socket`. |

| EVENT | FIRED ON | WHEN |
| --- | --- | --- |
| `connect` | `client · socket` | The client is connected. Also fires again after every reconnect. |
| `disconnect` | `both · socket` | The connection closed. The handler receives a `reason` string such as `transport close` or `ping timeout`. |
| `disconnecting` | `server · socket` | Just before disconnect, while `socket.rooms` is still filled. Useful for "user left the room" messages. |
| `connect_error` | `client · socket` | The connection attempt failed: server down, CORS rejected, or auth middleware refused. |

⚠️ **Do not use these names for your own events:** `connect`, `connect_error`, `disconnect`, `disconnecting`, `newListener`, `removeListener`. They are reserved.

💡 **Name events as `resource:action`**, for example `message:send`, `message:new`, `typing:start`, `room:join`. With thirty events in a project, this is what keeps them readable. And in React, remove each listener in the effect cleanup with `socket.off(name, handler)`, or the handler runs twice after a re-render.

The client emits `"message:send"` but the server is listening for `"sendMessage"`. What happens?

A — The server throws an "unknown event" error

B — Nothing. The event arrives, no handler matches, and no error is shown

Which of these is a reserved event name that you must not emit yourself?

A — disconnect

B — message:new

# 05 How Events Communicate

Who receives an event depends on what you call emit on

The event name decides **which handler** runs. The object you call `emit` on decides **who receives it**. On the client there is only one choice, the server. On the server there are several.

## The emit cheat sheet (server side)

| CALL | REACHES | TYPICAL USE |
|---|---|---|
| `socket.emit()` | only this client | Reply to the sender: their own data, an error, a welcome message. |
| `io.emit()` | every client | Global announcement, live visitor count. |
| `socket.broadcast.emit()` | everyone except sender | "Riya came online". Riya already knows. |
| `io.to(room).emit()` | everyone in the room | A system message in a chat room. |
| `socket.to(room).emit()` | room, except sender | A new chat message, a typing indicator. |
| `io.to(socketId).emit()` | one specific client | Private message. Every socket is automatically in a room named after its own ID. |

## Rooms

> KEY CONCEPT
>
> A **room** is a named group of sockets that exists only on the server. A socket enters with `socket.join(name)` and leaves with `socket.leave(name)`, and it leaves every room automatically when it disconnects. The client cannot join a room by itself. It emits an event asking to join, and the server decides.

SERVER COPY

```
io.on("connection", (socket) => {
  socket.on("room:join", (roomId) => {
    // check here that this user is allowed in this room
```

```
    socket.join(roomId);
    socket.to(roomId).emit("room:user-joined", { id: socket.id });
  });

  socket.on("typing:start", (roomId) => {
    socket.to(roomId).emit("typing:start", { id: socket.id });
  });
});
```

💡 **Give every user a personal room.** On connect, call `socket.join("user:" + userId)`. A user with three tabs open has three sockets, and `io.to("user:" + userId).emit(...)` reaches all three without you tracking socket IDs.

## Acknowledgements

A normal `emit` is fire and forget: you do not know whether the other side processed it. Pass a function as the **last argument** and Socket.IO turns the event into a request with a response. The receiver gets that function as its last parameter and calls it with the result.

CLIENT COPY

```
socket.emit("message:send", { roomId, text }, (response) => {
  if (response.ok) markAsSent(response.id);
  else showError(response.error);
});

// with a timeout, so the UI never waits forever
socket.timeout(5000).emit("message:send", { roomId, text }, (err, response) => {
  if (err) showError("Server did not respond in 5 seconds");
});
```

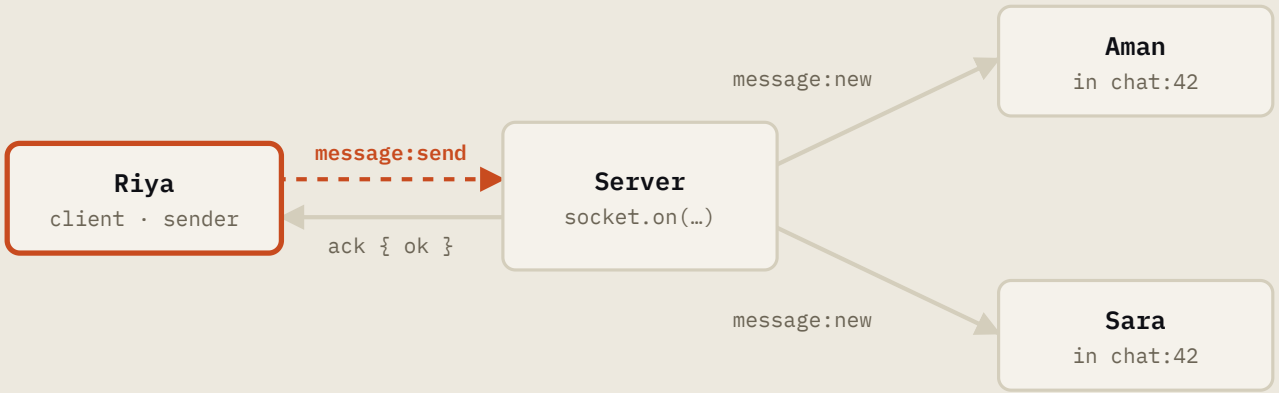
SERVER COPY

```
socket.on("message:send", async (payload, ack) => {
  try {
    const message = await Message.create({
      room: payload.roomId,
      text: payload.text,
      from: socket.data.userId,
    });
```

```
    ack({ ok: true, id: message.id });
    socket.to(payload.roomId).emit("message:new", message);
  } catch (err) {
    ack({ ok: false, error: "Could not save the message" });
  }
});
```

## The full round trip of one chat message

Riya, Aman and Sara are all in room `chat:42`. Riya sends a message. Use Back and Forward to walk through what happens.




### Riya emits the event

Her client calls socket.emit("message:send", payload, ack). The packet travels over her open connection to the server.

← BACK  Step 1 / 5  FORWARD →

Riya sends a chat message. Everyone in the room should get it except Riya, who already shows it in her own UI. Which call is correct?

A — socket.to(roomId).emit("message:new", message)

B — io.to(roomId).emit("message:new", message)

How does a client join a room?

A — It calls socket.join(roomId) in the browser

B — It emits an event, and the server calls socket.join(roomId) for it

## 06 Middleware

A checkpoint every client passes before the connection is accepted

So far anyone who knows your server URL can connect and start emitting events. In Express you solve this with middleware that runs before the route handler. Socket.IO has the same idea, with one important difference in **when** it runs.

> KEY CONCEPT
>
> A Socket.IO **middleware** is a function registered with `io.use()` that receives `(socket, next)`. It runs **once per connection**, during the handshake, before the `connection` event. Call `next()` to let the client in, or `next(new Error(...))` to refuse it.

- **Client asks to connect**
  It sends its credentials along with the handshake.
- **Middlewares run in the order you registered them**
  Each one must call `next()`. If one calls `next(err)`, the rest are skipped and the client receives `connect_error`.
- **`io.on("connection")` fires**
  Only clients that passed every middleware reach your connection handler.

**STEP 1 — SEND THE TOKEN FROM THE CLIENT**

CLIENT COPY

```
const socket = io("http://localhost:3000", {
  auth: { token: accessToken },
```

```
});

socket.on("connect_error", (err) => {
  console.log(err.message);   // "unauthorized"
});
```

**STEP 2 — VERIFY IT IN A MIDDLEWARE**

SERVER COPY

```
import jwt from "jsonwebtoken";

io.use((socket, next) => {
  try {
    const token = socket.handshake.auth.token;
    const payload = jwt.verify(token, process.env.JWT_SECRET);

    socket.data.userId = payload.sub;   // available in every handler
    next();
  } catch (err) {
    next(new Error("unauthorized"));
  }
});

io.on("connection", (socket) => {
  // only authenticated clients reach this point
  socket.join("user:" + socket.data.userId);
});
```

| PIECE | VALUE USED | WHAT IT DOES |
|---|---|---|
| **auth** | `{ token }` | Client option. Whatever you put here is sent with the handshake. |
| **socket.handshake.auth** | `{ token }` | Where the server reads it. `socket.handshake.headers` holds the cookies and other HTTP headers. |
| **socket.data** | `{ userId }` | A free object on each socket. Store who this connection belongs to, and read it later in any event handler. |

| PIECE | VALUE USED | WHAT IT DOES |
| --- | --- | --- |
| `next()` | `no argument` | Pass to the next middleware, or to the `connection` event if this was the last one. |
| `next(err)` | `new Error("…")` | Refuse the connection. The client gets `connect_error` with the same message. |

⚠️ **Always call `next`.** If a middleware returns without calling `next()` or `next(err)`, the client is left waiting until its connection attempt times out.

## The difference from Express middleware

| What | Express middleware | Socket.IO middleware |
| --- | --- | --- |
| **Signature** | (req, res, next) | (socket, next) |
| **Runs** | On every request | Once, when the client connects |
| **To reject** | res.status(401).json(…) | next(new Error(…)) |
| **Client sees** | A 401 response | A connect_error event |

⚠️ **`io.use()` does not run for each event.** It proves who the user is, once. It does not prove that this user may join room `chat:42` or delete a message. Check permissions inside the event handler every time.

## Middleware for every incoming event

When you want a check on each event, for example logging or rate limiting, register a middleware on the socket itself with `socket.use()`. It receives the event name and its arguments as an array.

SERVER COPY

```
io.on("connection", (socket) => {
  socket.use(([eventName, ...args], next) => {
    console.log(socket.data.userId, "sent", eventName);

    if (isRateLimited(socket.data.userId)) {
      return next(new Error("too many events"));
```

```
    }
    next();
  });

  // an error passed to next() arrives here, the event handler is skipped
  socket.on("error", (err) => {
    socket.emit("app:error", { message: err.message });
  });
});
```

💡 **Every reconnect runs the middleware again**, because a reconnect is a new connection. If the access token expired in the meantime, set a fresh one with `socket.auth = { token: newToken }` on the client before it reconnects.

A client connects once and then emits 50 events. How many times does the `io.use()` middleware run for that client?

A — Once, during the connection handshake

B — 51 times, once for the connection and once per event

A middleware calls `next(new Error("unauthorized"))`. What does the client receive?

A — A connect event followed by a disconnect event

B — A connect_error event whose message is "unauthorized"

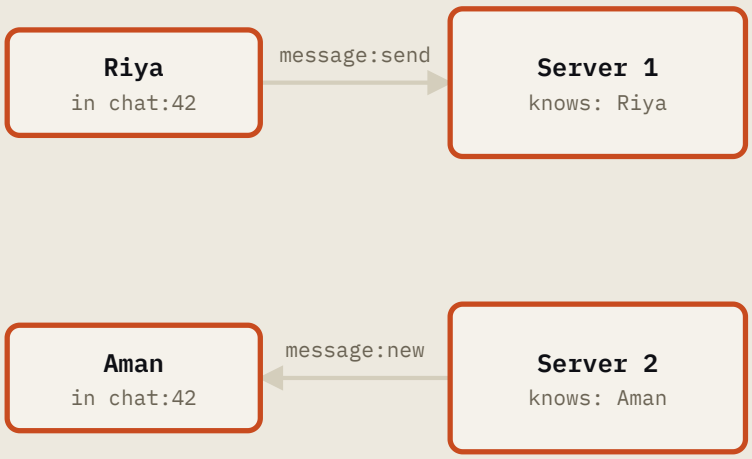
## 07 Problems While Scaling

Everything above works on one server. Here is what breaks on two.

A REST API scales easily: requests are short and stateless, so any server can answer any request. A socket connection is the opposite. It is **long-lived** and it belongs to **one specific server process**. Every scaling problem with Socket.IO comes from that one fact.

## Problem 1: Events do not cross servers

Rooms live in the memory of each server process. Server 1 only knows about the sockets connected to Server 1.




**Two servers behind a load balancer**

The load balancer sent Riya's connection to Server 1 and Aman's to Server 2. Both joined room chat:42, but each server only recorded its own socket.

← BACK Step 1 / 6 FORWARD →

FIX — REDIS ADAPTER

An **adapter** is the part of Socket.IO that delivers an emit to sockets. The default one only looks in local memory. The Redis adapter also publishes every emit to Redis, and every server is subscribed, so each server delivers to its own local sockets.

TERMINAL COPY

```
npm install @socket.io/redis-adapter redis
```

SERVER.JS COPY

```ts
import { createAdapter } from "@socket.io/redis-adapter";
import { createClient } from "redis";

const pubClient = createClient({ url: process.env.REDIS_URL });
const subClient = pubClient.duplicate();

await Promise.all([pubClient.connect(), subClient.connect()]);

io.adapter(createAdapter(pubClient, subClient));

// nothing else changes: this now reaches the room on every server
io.to("chat:42").emit("message:new", message);
```

⚠️ **The Redis adapter forwards events, it does not store them.** If Aman is offline when the message is emitted, he never receives it. Chat history must be saved in your database and fetched when the client connects again.

## Problem 2: The handshake breaks without sticky sessions

Remember that a connection starts on long polling, which is several separate HTTP requests. If the load balancer sends the first request to Server 1 and the next to Server 2, Server 2 has never seen that session ID and answers `400` with `Session ID unknown`. The client keeps failing and reconnecting.

| Fix | How | Trade-off |
|---|---|---|
| **Sticky sessions** | Configure the load balancer to always route one client to the same server (cookie or IP based). | Keeps the long-polling fallback. Needs load balancer configuration. |
| **WebSocket only** | On the client: `io(URL, { transports: ["websocket"] })`. One connection, so there is nothing to route twice. | No sticky sessions needed. Users on networks that block WebSocket cannot connect. |

## Problem 3: State kept in server memory

A `const onlineUsers = new Map()` at the top of your file works on one server. On three servers you have three different maps, each with a third of the users. Anything that more than one server must agree on belongs in Redis or the database: online status, who is in which room, unread counts.

## Problem 4: Every restart drops connections

A deploy restarts the process, and every socket on it closes. All of those clients reconnect at the same moment, which can overload the servers that are still running.

- **Shut down gracefully.** On `SIGTERM`, call `io.close()` so clients are disconnected cleanly and reconnect to another server.
- **Roll the deploy.** Restart one server at a time so only a fraction of users reconnect together.
- **Catch up on reconnect.** The client sends the ID of the last message it saw, and the server returns everything after it from the database.

SERVER.JS COPY

```
process.on("SIGTERM", () => {
  // disconnects every socket and closes the HTTP server
  io.close(() => process.exit(0));
});
```

## Problem 5: Idle connections still cost memory

An idle REST user costs the server nothing. An idle socket user holds an open connection, a file descriptor and some memory the whole time. Fifty thousand quiet users are still fifty thousand open connections, so you plan capacity by **connected users**, not by requests per second.

SUMMARY

| Problem | Cause | Fix |
| --- | --- | --- |
| **Events reach only some users** | Rooms live in one process | Redis adapter |
| **400 Session ID unknown** | Polling requests hit different servers | Sticky sessions, or WebSocket only |
| **Wrong online counts** | State in a local variable | Move shared state to Redis or the DB |
| **Reconnect storm on deploy** | Restart closes every socket at once | Graceful shutdown + rolling deploy |

| Problem | Cause | Fix |
| --- | --- | --- |
| **Missed messages** | Events are not stored anywhere | Save to DB, catch up on reconnect |

You run two Socket.IO servers with no adapter. Riya is on Server 1 and Aman on Server 2, both in room `chat:42`. Server 1 runs `io.to("chat:42").emit(...)`. Who receives it?

A — Riya and Aman, because they joined the same room

B — Only Riya, because Server 1 only knows its own sockets

With the Redis adapter installed, Aman is offline when a message is emitted to his room. What happens when he comes back?

A — He gets nothing from Redis. The app must load missed messages from the database

B — Redis kept the message and delivers it when he reconnects

---